

게이팅 단일화 — 계획 + 명세

세 곳에 흩어진 게이트와 두 별로 평행 존재하는 쿨 상태를 **AbilityRunner** 한 권위로. 쿨은 절대시각으로, 코스트만 Stat에 위임.

한 줄 요약 — 지금 세 곳(`CharacterCombat` 평타레이트 · `CharacterStat` Q/E 쿨 · `AbilityRunner` 쿨)에 흩어진 게이트와, **두 곳(Stat-Runner)에 평행 존재하는 쿨 상태**를 `AbilityRunner` 한 권위로 모은다. 쿨은 절대시각(IClock)으로만 존재 → `PartyManager.TickCooldowns` 삭제. 코스트(에너지·궁게이지)만 Stat에 남기고 러너가 **위임 호출**로 소비한다.

상위 청사진: `Player_Refactor_Blueprint §02-§04` · 선행: `Phase0_Result.md` · **범위** — Phase 1만. 발동 경로 통일 (E→AbilityBeat)은 Phase 2, 컷신/Q 분리는 Phase 3. 여기서 fire 타이밍을 바꾸지 않는다.

01 지금 상태 — 무엇이 어디에

게이트 3주인 · 쿨 상태 2벌 평행 · delta-tick의 우회비용.

게이트	위치	판단	상태 소유	시각
평타 레이 트	<code>CharacterCombat.CanAttack</code>	<code>Now - _lastAttackTime ≥ _attackRate</code>	<code>_lastAttackTime</code>	<code>Time.time</code>
Q 쿨 +궁	<code>CharacterStat.TryUseSkillQ</code>	<code>QCool≤0 AND Energy≥Max</code>	<code>CurrentQSkillCoolTime</code>	delta-tick
E 쿨	<code>CharacterStat.TryUseSkillE</code>	<code>ECool≤0</code>	<code>CurrentESkillCoolTime</code>	delta-tick
러너 쿨 +태 그	<code>AbilityRunner.TryCast</code>	<code>IsOnCooldown + Req/Blocked 태그</code>	<code>_cooldownEnd[AbilityData]</code>	<code>IClock.Now</code>

중복의 핵심 — 같은 Q/E 발동이 **Stat 게이트(입력 시점)** 와 **러너 게이트(fire 시점)** 를 둘 다 통과한다. 쿨 상태가 `Current*CoolTime` (delta-tick)과 `_cooldownEnd` (절대시각) 두 별로 평행 존재 → "이 스킬 쿨 누가 정함?"이 모호.

delta-tick의 대가 — 스왑아웃(비활성) 멤버는 `CharacterStat.Update` 가 안 돌아 쿨이 멈춘다. 이를 메우려 `PartyManager.TickCooldowns(dt)` 가 매 프레임 전 멤버를 대신 tick한다. **절대시각으로 가면 이 우회가 통째로 불필요.**

발동 흐름 (fire 타이밍 — Phase 1에서 유지)

슬롯	입력 진입점	입력 시점 게이트(현재)	상태 전이	fire(이펙트) 시점
평타	Attack	<code>_combat.CanAttack</code> + <code>TryAttack</code>	→ Attack	애니 <code>AbilityBeat</code> → <code>UseAbility(Attack, beat)</code>
Q	<code>UseSkillQ</code>	<code>_combat.TryUseSkillQ (=Stat)</code>	→ CutScene→Q_Skill	애니 <code>AbilityBeat</code> → <code>UseAbility(Skill, beat)</code>
E	<code>UseSkillE</code>	<code>_combat.TryUseSkillE (=Stat)</code>	→ E_Skill	즉발 <code>UseAbility(Ex)</code> (입력 시점)

게이트의 두 역할

(A) 입장(admission) — 스킬 상태로 들어갈지(입력 시점) · (B) 발사(fire) — 이펙트 실행(비트/즉발). Phase 1의 과제는 A·B 둘 다 러너가 소유하는 쿨/코스트/태그를 보게 만드는 것. fire 타이밍 자체는 안 건드린다.

02 목표 설계 — 러너가 admission과 fire를 분리 소유

`TryCast` 한 덩어리(게이트+코스트+쿨+캐스트+이펙트)를 admission(동기·부작용)과 fire(비동기·무게이트)로 쪼갬다.

2.1 AbilityRunner 새 공개 API (명세)

```
public class AbilityRunner
{
    // — 판독 (부작용 없음) — UI/스펙 체크. 아무것도 소비/변경 X —
    public bool CanCast(AbilityData a, AbilityContext ctx);

    public float CooldownRemaining(AbilityData a); // 남은 쿨(초). 준비되면 0
    public float CooldownDuration(AbilityData a); // 이번 쿨 총 길이(CDR 반영값 저장 → UI 정확)

    // — admission (동기·커밋) — 통과 시 코스트 소비 + 쿨 시작. 이펙트는 안 함 —
    public bool Commit(AbilityData a, AbilityContext ctx);

    // — fire (비동기·무게이트) — 캐스트타임 → 이펙트 순차 → GrantsTags try/finally —
    public UniTask Fire(AbilityData a, AbilityContext ctx, CancellationToken token);

    // — 편의: Commit+Fire 동시. 몬스터/보스처럼 게이트·발사가 같은 순간인 호출부용 —
    // 기존 시그니처 유지 → BaseMonsterController 등 호출부 무변경
    public async UniTask TryCast(AbilityData a, AbilityContext ctx, CancellationToken token)
    {
        if (!Commit(a, ctx)) return;
        await Fire(a, ctx, token);
    }
}
```

책임 분해

- `Commit`: ① 쿨 검사 → ② 코스트 검사·소비(\$3) → ③ 태그 게이트(Req/Blocked) → ④ 쿨 시작
(`_cooldownEnd = Now + Cooldown·CDR`). 하나라도 실패 시 부작용 0으로 조용히 false.

- `Fire` : 캐스트타임 대기 → `BuildRuntimeEffects()` 순차 실행 → `GrantsTags` (캐스트 스코프, 예 `State.Casting`) try/finally 해제. 쿨/코스트/Req-Blocked 재검사 없음.
- `GrantsTags` 는 fire 소유(발동 지속), `RequiredTags` / `BlockedTags` 는 admission 소유(입장 판정).

2.2 게이트 순서 (Commit 내부 — 청사진 §04와 동일)

- 1 쿨다운 — `IClock.Now < _cooldownEnd[a]` 이면 아직 쿨 → 중단 (절대시각, tick 불필요)
 - 2 코스트 — `resource.CanPay(a)` 부족하면 중단(미소모). 통과 시 `resource.Pay(a)`
 - 3 태그 — `HasAll(Required) && HasNone(Blocked)` (기절 중 발동 불가 등)
 - 4 쿨 시작 — `_cooldownEnd[a] = Now + Cooldown·CDR` → (Fire에서 캐스트타임 + 이펙트)
- ✗ 1~3 실패 시 — 상태변화 없이 조용히 중단, 쿨·코스트 미소모

순서 결정 필요

코스트 소비를 태그 게이트 앞에 두면 태그 실패 시 궁게이지가 샌다. 안전하게 "검사 전부 통과 → 마지막에 Pay + 쿨시작" 으로 짜길 권장 (§8-c).

03 코스트 이음새 — 유일하게 새로 잇는 배선

에너지는 Stat에 남고, 러너는 소비 순서만 소유. `AbilityData.Cost` 는 지금 죽은 필드 — 여기가 실제 배선점.

3.1 비용 종류 (Q의 "궁게이지" 의미 보존)

Q의 코스트는 정수가 아니라 "에너지 만충 요구 + 전량 소비". E-평타는 없음. 이를 담으려면 `AbilityData` 에 종류 필드를 둔다.

```
public enum EAbilityCostType { None, Energy, UltimateGauge }

// AbilityData 에 추가
public EAbilityCostType CostType = EAbilityCostType.None;
// Cost(int) 는 CostType==Energy 일 때만 의미 (기존 필드 재사용)
```

슬롯	CostType	CanPay	Pay
평타 / E	None	항상 true	no-op
Q	UltimateGauge	<code>CurrentEnergy ≥ MaxEnergy</code>	<code>CurrentEnergy = 0</code>
(미래 스킬)	Energy	<code>CurrentEnergy ≥ Cost</code>	<code>CurrentEnergy -= Cost</code>

3.2 자원 제공자 인터페이스

`AbilityContext.CasterStat` 은 `BaseStat` . 코스트는 캐릭터 전용이라 인터페이스로 얇게 엮고 러너는 널체크한다(몬스터 미구현 → 코스트 없음).

```

public interface IAbilityResource
{
    bool CanPay(AbilityData a);
    void Pay(AbilityData a); // 호출 후 OnEnergyChanged / 궁 준비 재평가는 구현체 책임
}
// CharacterStat : BaseStat, IAbilityResource → 위 표대로 구현
// 러너 Commit 내부:
var res = ctx.CasterStat as IAbilityResource;
if (res != null && !res.CanPay(a)) return false;
// ... 전 게이트 통과 후 ... res?.Pay(a);

```

04 슬롯별 admission / fire 매핑 (Phase 1)

규약 — 슬롯의 쿨·코스트 대표 어빌리티는 `_slotMap[slot][0]`. 비트는 각자 `list[beat]` 이펙트만 발사(무게이트).

슬롯	admission (입력)	fire
평타	<code>Commit(atk0)</code> 로 Attack 입장 게이트	각 비트 → <code>Fire(list[beat])</code>
Q	<code>Commit(q0)</code> 로 컷신 입장 (궁게이지 여기서 소비)	각 비트 → <code>Fire(list[beat])</code>
E	<code>TryCast(ex0)</code> 입력 시점 즉발(Commit+Fire) — Phase 1 경로 유지	— (Phase 2에서 비트로)

`atk0=_slotMap[Attack][0]`, `q0=_slotMap[Skill][0]`, `ex0=_slotMap[Ex][0]`.

BaseCharacter 진입점 재작성 (명세)

```

// 평타
public void Attack(bool isPressing) {
    if (!isPressing) return;
    if (!_stateMachine.CanAttack || IsUsingSkill || Stat.HealthComp.IsDead) return;
    if (!TryCommitSlot(CharacterAbilitySlot.Attack)) return; // ← _combat.CanAttack/TryAttack 대체
    _stateMachine.ChangeState(PlayerState.Attack);
}
// Q
public void UseSkillQ() {
    if (!_stateMachine.CanUseSkill || IsUsingSkill || Stat.HealthComp.IsDead) return;
    if (!TryCommitSlot(CharacterAbilitySlot.Skill)) return; // ← _combat.TryUseSkillQ 대체 (궁+쿨)
    _stateMachine.ChangeState(PlayerState.QSkillCutScene);
}
// E (Phase 1: 즉발 유지)
public void UseSkillE() {
    if (!_stateMachine.CanUseSkill || IsUsingSkill || Stat.HealthComp.IsDead) return;
    if (!CanCastSlot(CharacterAbilitySlot.Ex)) return; // 입장 판정만(소비 X)
    _stateMachine.ChangeState(PlayerState.E_Skill);
    FireSlotImmediate(CharacterAbilitySlot.Ex); // TryCast(ex0)=Commit+Fire
}
// 비트: 이펙트만 (무게이트)
public void UseAbility(CharacterAbilitySlot slot, int beat = 0) {
    if (_slotMap.TryGetValue(slot, out var list) && beat ≥ 0 && beat < list.Count)
        FireEffectsOnly(list[beat]); // ← 기존 TryCast(list[beat]) 를 Fire 로 교체
}

```

- `TryCommitSlot(slot)` = `_slotMap[slot]?[0]` 로 ctx 만들어 `_abilityRunner.Commit(a, ctx)`.

- `FireEffectsOnly(a)` = ctx+token 만들어 `_abilityRunner.Fire(a,ctx,token).Forget()`.
- E도 `TryCommitSlot` + `FireEffectsOnly` 로 통일해도 됨(권장, 경로 일관).

다중 비트 자기차단 주의 (덤으로 잡히는 버그)

지금 비트마다 `TryCast(list[beat])` 라 같은 어빌리티를 2비트에서 쓰면 두 번째가 자기 쿨에 막혀 조용히 씹힌다. 비트를 **무게이트 Fire** 로 바꾸면 이 잠복 버그도 사라진다(쿨은 admission 한 번만).

05 UI 읽기 — Stat 쿨 필드 → 러너 조회

HUD/슬롯UI가 읽던 `stat.CurrentQSkillCoolTime` 가 사라진다. BaseCharacter 얇은 파사드로 러너를 조회.

```
// BaseCharacter (읽기 파사드)
public float SlotCooldownRemaining(CharacterAbilitySlot s); // 러너 CooldownRemaining(slot0)
public float SlotCooldownDuration(CharacterAbilitySlot s); // 러너 CooldownDuration(slot0)
public bool IsUltimateReady => // 궁 준비 = 에너지 만충 && Q 쿨 0
    Stat.CurrentEnergy >= Stat.MaxEnergy.Value
    && SlotCooldownRemaining(CharacterAbilitySlot.Skill) <= 0f;
```

바꿀 곳

- `ActiveCharacterHUD.UpdateCooldowns(stat)` → `UpdateCooldowns(BaseCharacter c)` 로 변경, `c.SlotCooldownRemaining/Duration(Skill|Ex)` 로 채움. 호출부 `GameSceneCanvas.Update` 는 이미 매 프레임 폴링 중.
- `SubscribeEvent / PartySlotUI` 의 `isReady` 계산 → `c.IsUltimateReady`.

궁 준비 이벤트(ONULTIMATESTATECHANGED) 처리 — 결정 포인트

쿨이 절대시각(무tick)이 되며 "쿨 방금 0 됨" 전이를 쳐줄 주체가 없어진다. **활성 캐릭터는**

`GameSceneCanvas.Update` 폴링에서 `c.IsUltimateReady` 계산해 푸시. **벤치 멤버는** §6의 경량 ready 재평가 패스가 전이 시 이벤트를 계속 쳐 주면 슬롯UI 변경 최소화.

(a) 이벤트 유지 + 소스만 러너로 (권장) vs (b) 이벤트 제거 + 전부 폴링.

06 PartyManager.TickCooldowns 처리

- 쿨 진행 목적은 삭제 — 쿨이 절대시각이라 벤치 멤버도 `Now` 가 흐르면 자동 경과. `Stat.TickCooldowns / Current*CoolTime / QSkillCoolTime / ESkillCoolTime (Stat)` 전부 제거.
- 대체(선택, 5-(a) 채택 시) — 그 자리에 **궁 준비 재평가 패스**만 남긴다. dt 계산 없이 각 멤버 `IsUltimateReady` 를 재계산해 이전 값과 다르면 `OnUltimateStateChanged` 를 친다(멤버당 bool 비교뿐, 기존보다 가벼움). 이름 `RefreshUltimateReady()` 등.
- `Managers.Update` 의 `TickCooldowns(dt)` 호출부도 교체/제거.

07 파일별 변경 체크리스트

신규 · 수정 · 삭제. 커밋 경계는 §10.

신규 **NEW**

- `Assets/Scripts/Ability/IAbilityResource.cs` — `CanPay` / `Pay` 인터페이스
- (선택) `EAbilityCostType` 는 `AbilityData.cs` 안에 뒤도 됨

AbilityRunner.cs **유지·강화**

- `TryCast` 분해 → `CanCast` / `Commit` / `Fire` 추가, `TryCast` = `Commit` + `Fire` 편의로 남김(시그니처 유지)
- `CooldownRemaining` / `CooldownDuration` 공개
- `Commit` 에 코스트 이음새(`ctx.CasterStat as IAbilityResource`). 쿨 길이 저장 시 CDR 곱 지점 마련(값은 Phase 4)
- 태그: `Req/Blocked` → `Commit` , `Grants` → `Fire` (try/finally 유지)

AbilityData.cs **필드 추가**

- `EAbilityCostType CostType` 추가. `Cost` (int)는 Energy용 재사용
- Q SO: `CostType=UltimateGauge` · E/평타 SO: `None`
- **데이터 이관** — `CharacterDataSO.QSkillCoolTime/ESkillCoolTime` · `_attackRate` (0.5) 값을 각 슬롯 대표 어빌리티 `Cooldown` 으로 복사(디자인)

CharacterStat.cs **축소**

- `IAbilityResource` 구현(`CanPay` / `Pay` — §3 표)
- 삭제: `QSkillCoolTime` / `ESkillCoolTime` (Stat), `Current*CoolTime` , `TickCooldowns` , `TryUseSkillQ` , `TryUseSkillE`
- `CheckUltimateReadyState` 의 쿨 참조 제거 → 궁 준비는 러너 쿨 기준(§5). 에너지 만충 판정만
- `SetCharacterData` / `ResetState` / `ReturnToTownState` 의 `Current*CoolTime` 초기화 제거. 쿨 리셋 필요 시 `AbilityRunner.ResetCooldowns()`
- `AddEnergy` ·에너지 regen(`Update`)· `OnEnergyChanged` 는 유지

CharacterCombat.cs **해체·삭제**

- `CanAttack` / `TryAttack` / `TryUseSkillQ` / `TryUseSkillE` 전부 러너/Stat로 흡수 → 파일 제거

BaseCharacter.cs **배선 변경**

- `_combat` 필드·생성 제거. `_attackRate` 직렬 필드는 어빌리티 `Cooldown` 으로 이관 후 제거
- `Attack` / `UseSkillQ` / `UseSkillE` → §4 명세대로 `Commit` / `CanCast` / `Fire`
- `UseAbility` (비트) → `TryCast(list[beat])` 대신 `Fire(list[beat])`
- 읽기 파사드 `SlotCooldownRemaining/Duration` , `IsUltimateReady` 추가

PartyManager.cs / Managers.cs **tick 제거**

- `TickCooldowns` 제거 또는 `RefreshUltimateReady` 로 대체(§6). `Managers.Update` 호출부 갱신

- ☐ UpdateCooldowns 가 BaseCharacter 파사드를 읽도록
- ☐ isReady 계산을 c.IsUltimateReady 로
- ☐ (5-(a) 채택 시) OnUltimateStateChanged 구독 유지 — 소스가 러너 쿨

08 엡지케이스 & 결정 포인트

#	사안	처리
a	다중 비트 대표 어빌리티	슬롯 쿨/코스트는 list[0] 만. 비트는 무게이트 Fire → 자기차단 버그 소멸
b	절대시각 쿨 + 전투 재시작	러너 쿨은 timestamp라 리셋하려면 AbilityRunner.ResetCooldowns() 필요. 매 판 쿨 초기화가 스펙이면 BaseCharacter 리셋 경로에서 호출(누락 시 이전 판 쿨 잔류)
c	코스트 vs 태그 소비 순서	"검사 전부 통과 → 마지막에 Pay + 쿨시작"으로 태그 실패 시 궁게이지 새는 것 방지
d	벤치 멤버 에너지	지금도 Stat.Update 비활성이라 벤치 에너지 안 참(변경 없음). 쿨만 절대시각 자동 경과 — 의도 일치
e	몬스터/보스	TryCast 시그니처 유지 → 무변경. 코스트 인터페이스 미구현 → as IAbilityResource==null → 코스트 없음
f	CDR(쿨감)	_cooldownEnd = Now + Cooldown·CDR 곱 지점만 마련. 실제 배선은 Phase 4(여기선 ×1.0)
g	E의 CanCast→Fire 창	Phase 1 E는 즉발이라 창 없음. Phase 2에서 비트로 가면 admission-fire가 벌어지므로 그때 Q와 동일 패턴

09 검증 시나리오

- ☐ **평타** — 연타 시 어빌리티 Cooldown (구 attackRate) 간격으로만 발동. 상태/이펙트 정상
- ☐ **Q** — 에너지 만충에서만 발동, 발동 즉시 에너지 바 0, Q 쿨 게이지 채워짐(러너 조회). 쿨 중 재입력 씹힘
- ☐ **E** — E 쿨 간격 준수(구 ESkillCoolTime). 연타 방지 유지
- ☐ **스왑아웃 쿨 진행** — Q/E 쓰고 스왑아웃 → 시간 경과 후 스왑인 시 쿨이 절대시각만큼 경과. TickCooldowns 없이 동작
- ☐ **궁 준비 표시** — 활성 HUD·벤치 슬롯UI 모두 (에너지 만충 && Q 쿨 0)에서 ready 점등. 쿨 완료 순간 전이 반영
- ☐ **쿨 UI 채움** — HUD Q/E 아이콘 fill이 1 - Remaining/Duration 로 매끄럽게
- ☐ **다중 비트** — 한 평타/Q 애니의 여러 비트가 전부 발사(자기차단 없음)
- ☐ **재진입/전투 재시작** — 쿨 초기화 정책(§8-b) 의도대로
- ☐ **몬스터 회귀 없음** — 몹 어빌리티 발동·쿨·태그 이전과 동일

10 커밋 경계 — 독립 배포 가능하게

커밋 1 씨앗 (동작 불변)

`IAbilityResource` + `AbilityData.CostType` + `AbilityRunner` API 분해(`CanCast/Commit/Fire` , `TryCast` =편의). 아직 아무도 새 API 안 씀. 몬스터 회귀 없음 확인.

커밋 2 코스트 이관

`CharacterStat` 가 `IAbilityResource` 구현, `Commit` 에 코스트 이음새. Q 궁게이지 러너 경유로.

커밋 3 admission 이관 + CharacterCombat 삭제

`Attack/UseSkillQ/UseSkillE` 를 러너 게이트로, `UseAbility` 비트를 `Fire` 로. `CharacterCombat` 제거.

커밋 4 UI 이관

HUD/슬롯UI가 러너 파사드 읽기. `OnUltimateStateChanged` 소스 러너로.

커밋 5 tick 제거

`Stat.Current*CoolTime / TickCooldowns` 삭제, `PartyManager.TickCooldowns` → `RefreshUltimateReady` (또는 제거).

롤백면

각 커밋 뒤 에디터 플레이 실측. 커밋 3에서 회귀 시 게이트만 되돌리면 됨 — 상태 소유가 러너로 단일화돼 롤백면이 좁다.

다음 — PHASE 2 (발동 경로 통일)

E를 `AbilityBeat` (Ex 슬롯)로 옮겨 `admission(Commit)`과 `fire(beat)`를 Q·평타와 같은 경로. `SlotForState` 에 `E_Skill` → Ex 한 줄.